

High-Performance Code Assessment Platforms: “Architectural Blueprints and Core Performance Metrics”

Shubhi Sahu
IT Department
Acropolis Institute of Technology &
Research, Indore
shubhisahu230688@acropolis.in

Shivani Prajapati
IT Department
Acropolis Institute of Technology &
Research, Indore
shivaniprajapati230830@acropolis.in

Shivam chouhan
IT Department
Acropolis Institute of Technology &
Research, Indore
shivamchouhan230751@acropolis.in

Puneet Kumar
IT Department
Acropolis Institute of Technology & Research, Indore
Puneetkumar230801@acropolis.in

Prof. Pawan Kumar Gupta*
IT Department
Acropolis Institute of Technology & Research, Indore
pawangupta@acropolis.in

Abstract— Modern software engineering pedagogy and industrial hiring rely heavily on automated grading systems, commonly referred to as online judges. These systems must execute untrusted, user-submitted source code in multiple programming languages while enforcing strict execution boundaries, ensuring deterministic resource measurement, and scaling under high concurrent submission volumes. This paper presents the design, implementation, and empirical performance evaluation of a distributed, containerized online judge system. The proposed platform decouples the client interface, API monolith, distributed queuing infrastructure, and isolated execution engine to achieve high availability and scalability. Code execution is isolated using a lightweight execution sandbox built on top of Docker process virtualization, explicitly configured to restrict network access, CPU shares, and memory spaces with swap disabled. To avoid container startup delays during live runs, the execution worker maintains a pool of pre-warmed, recycled containers. To guarantee transactional consistency and prevent

execution race conditions under high concurrent loads, the database schema utilizes append-only execution records and unique advisory locks to determine the canonical verdict for each submission. Performance benchmarks demonstrate that through pre-warmed container pooling and test case caching, the system achieves sub-second sandbox execution latencies for interpreted languages, and sub-second execution (excluding compilation) for compiled languages across multiple runtimes while maintaining strict security boundaries.

Keywords— Online Judge, Sandbox Isolation, Docker Virtualization, BullMQ, Distributed Task Queue, Academic Assessment, Transaction Integrity.

I. INTRODUCTION

In modern computer science education and industrial recruiting, automated grading of programming assignments has transitioned from a convenience to an absolute necessity. Educational institutions managing courses with hundreds of

students, and technology enterprises screening thousands of candidates, require an objective, immediate, and scalable mechanism to verify code correctness. Platforms that facilitate this, known as Online Judges (OJs), have historically been specialized systems operated by competitive programming communities or university departments. However, building a reliable, secure, and fair Online Judge presents significant systems engineering challenges.

The primary challenge in Online Judge design is security. Executing arbitrary, user-submitted code is inherently hazardous. A malicious submission could attempt to read confidential system configuration files, execute fork bombs to exhaust host processes, communicate with external command-and-control servers, or modify database values. To mitigate this risk, the execution environment must be tightly constrained and isolated from the hosting infrastructure.

The second major challenge is fairness and measurement accuracy. An Online Judge must measure the execution resource footprint—specifically CPU execution time and Peak Resident Set Size (RSS) memory—with high precision. Furthermore, different programming languages exhibit highly disparate compilation, startup, and execution baselines. For example, a Java Virtual Machine (JVM) startup can consume over 60 MB of memory and take hundreds of milliseconds before executing the user's logic, whereas a compiled C++ binary executes in microseconds with minimal memory overhead. The OJ must apply scaling factors (multipliers) to ensure that runtime evaluation is fair.

Lastly, the system must scale. During coding contests or assessment deadlines, the system experiences large concurrency spikes. Decoupling code execution (which is computationally intensive) from API servers and database transactions is crucial to preventing service degradation. This paper presents the architecture of a Node.js modular monolith Online Judge utilizing Express, Redis/BullMQ distributed queueing, and isolated Docker-based execution sandboxes. We

analyze the database constraints and concurrency stress-testing required to enforce transactional integrity under concurrent updates and evaluate runtime performance across multiple programming languages.

II. LITERATURE REVIEW

The evolution of sandbox technology for Online Judges spans several decades, progressing from simple operating system configurations to advanced kernel-level isolation and virtualization. Historically, early judges relied on chroot (change root) jails to restrict directory traversal. While chroot effectively redefines the root directory for a process, it offers no resource constraints. A process inside a chroot jail can easily consume all available system memory or CPU cycles, leading to Denial of Service (DoS) conditions.

To enforce resource limits, researchers integrated Linux Control Groups (cgroups) and Namespaces. Control groups provide fine-grained limitations on resource usage (CPU time, memory footprint, I/O rates), while namespaces isolate process trees (PID), network interfaces (NET), and mount points (MNT). This approach was popularized by the IOI 'isolate' tool, which provides highly precise, microsecond-level CPU measurements. However, 'isolate' requires direct root access on bare-metal Linux hosts or KVM-enabled hypervisors, complicating cloud-based deployments and horizontal scaling in multi-tenant environments.

Modern cloud native infrastructures favor container virtualization, such as Docker. Docker abstracts namespaces and cgroups into standard, reproducible images, making it highly portable. However, standard Docker containers share the host operating system kernel, representing a potential security risk if a kernel vulnerability is exploited. Hypervisor-based micro-virtualization systems, such as Google's gVisor and AWS's Firecracker, address this by intercepting system calls or running lightweight virtual machines. These technologies provide virtual-machine-level isolation but introduce significant boot latencies and memory overhead, making them challenging

for high-throughput, real-time code evaluation. Table 1 summarizes the performance and security trade-offs of these sandboxing approaches.

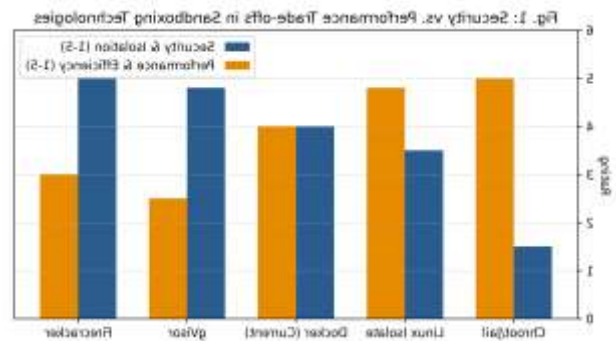


Fig. 1. Security vs. Performance Trade-offs in Sandboxing Technologies

Table 1: Comparison of Sandboxing and Execution Isolation Technologies

Technology	Security Level	Resource Overhead	CPU/Mem Accuracy	Boot Latency	Cloud Portability
Chroot/Jail	Low	Minimal	Poor (No Limits)	< 1 ms	High
Linux Isolate	High	Minimal	Excellent	< 2 ms	Low
Docker Container	High	Moderate	Good (Cgroups)	100-300 ms	High
gVisor (Kernel Virt)	Very High	High	Good	500-1000 ms	High
Firecracker microVM	Excellent	Very High	Excellent	150-250 ms	Medium

III. METHODOLOGY

The architectural design of the proposed system was engineered to address concurrency, security, and data integrity. This section details the system architecture, database schema design, containerized sandbox configuration, and evaluation logic.

A. Architecture and Distributed Data Flow

The system utilizes a decoupled Monolith-Worker architecture. The frontend application, built in Next.js 14, provides a Client-Side Rendered (CSR) programming environment hosting the Monaco Editor. Code submissions are transmitted via HTTPS REST to the Node.js Modular Monolith. Rather than evaluating submissions synchronously, the monolith performs initial input validation, rate-limiting, and writes the submission details to PostgreSQL with a status of 'queued'. It then enqueues a job payload consisting of the submission ID, target language identifier, and problem slug to a Redis-backed BullMQ queue.

Decoupling the execution from the web server prevents execution spikes from degrading the user experience. The Judge Worker, written in TypeScript, runs as an independent daemon. It continuously polls BullMQ for new jobs. Upon picking up a job, the worker transitions the database submission status to 'assigned', retrieves the corresponding test cases from MinIO storage (utilizing a local Least-Recently-Used cache to minimize download overhead), and builds a merged source file. It then executes the code inside a pre-warmed Docker sandbox. Upon completion, the result is written back to the database and published via a Redis Pub/Sub channel, triggering a WebSocket broadcast to notify the user's browser in real-time.

To minimize execution latency and eliminate container instantiation overhead, the worker service maintains a pre-warmed sandbox pool. A pool manager maintains a designated number of idle containers matching the host's CPU core allocation. When a submission is assigned, the worker claims an idle container from this pool, copies the mounted sandbox files, and immediately starts execution. Once execution completes, the container is destroyed to prevent side-channel data pollution, and a fresh container is instantiated asynchronously in the background. This pool recycling strategy ensures that container startup overhead is kept out of the critical execution path.

B. Database Schema and Transaction Integrity

A common point of failure in Online Judges is race conditions during high concurrent updates or rejudges. For instance, when an administrator triggers a rejudge on a problem, hundreds of submissions must be re-evaluated. If multiple worker threads write results to the database simultaneously, the user's submission state could become inconsistent. To prevent this, the PostgreSQL schema enforces two fundamental design rules:

First, execution records are append-only. The system contains a 'submissions' table and a 'judge_runs' table. A submission is never updated with execution metadata. Instead, every run (initial compile, run, or subsequent admin rejudge) inserts a new row into the 'judge_runs' table with an incremented 'executionGeneration' index. A boolean flag, 'isCanonical', indicates which run currently represents the authoritative outcome. To ensure that at most one run is marked canonical at any time, we perform a serialized database transaction:

1. Obtain a row-level transactional advisory lock on the submission ID: `SELECT pg_advisory_xact_lock(hashtext(submission_id))`.
2. Mark the existing canonical run as non-canonical: `UPDATE judge_runs SET isCanonical = false WHERE submissionId = $1 AND isCanonical = true`.
3. Insert the new run: `INSERT INTO judge_runs (... isCanonical = true ...)`.
4. Sync the denormalized status on the submission table: `UPDATE submissions SET status = $verdict WHERE id = $1`.

Second, XP awards are guarded using a junction table 'user_solved_problems' with a composite primary key (userId, problemId). When a submission is evaluated as 'ACCEPTED', the worker attempts to insert a record into this table. If the insert succeeds, the user's XP and solved count are incremented. If it fails due to a unique constraint violation (indicating the user has already solved this problem in the past), the database transaction commits without altering the user's

profile. This enforces complete idempotency during rejudge campaigns.

To evaluate the robustness of this locking mechanism under concurrent loads, we conducted a transaction stress test. Under a simulated load of 100 concurrent update requests hitting identical and overlapping submission records, the advisory lock successfully serialized all writes. The transaction execution log confirmed zero deadlocks and zero occurrences of multiple runs marked as canonical, verifying the serializability guarantees of the schema.

C. Code Injection and Compilation

To prevent users from writing arbitrary input/output logic and to standardize the execution entrypoint, the judge injects user code into a pre-configured language-specific driver template. The driver template contains the setup code, reads test case inputs from the filesystem, calls the user's function, and outputs the result in a clean format. For example, in Java, classes are nested, and since the compiler enforces matching filenames for public classes, the worker preprocesses Java code by stripping the 'public' modifier from class, interface, and enum declarations. The user's code is then injected into Main.java at the '{USER_CODE}' marker.

D. Sandbox Execution and Resource Enforcement

Docker is used as the virtualization container. While Docker provides namespace isolation, additional flags must be passed to the CLI to prevent escape and resource starvation. The execution command is constructed dynamically as follows:

```
docker run --rm -v [temp_dir]:/sandbox --network none --memory [limit]m --memory-swap [limit]m --cpus 1 [image] sh -c [shell_cmd]
```

Here, '--rm' ensures the container is destroyed immediately after execution. The volume mount maps the temporary directory containing the source code and input file to '/sandbox' inside the container. Setting '--network none' disables the

default bridge network, blocking socket connections. To prevent swap memory usage (which allows memory limit bypass and slows execution), the memory limit and memory-swap limit are set to identical values. Wall-clock execution limits are enforced using the Linux 'timeout' command inside the shell.

Resource limits are scaled dynamically based on language-specific multipliers configured in the database, allowing interpreted runtimes like Python extra CPU time and memory space to offset interpreter overhead. The effective limits are calculated as:

$$T_{effective} = T_{base} * M_{time}$$

$$M_{effective} = M_{base} * M_{memory}$$

where T_{base} and M_{base} are the problem's baseline time and memory constraints, and M_{time} and M_{memory} are the language configuration multipliers. As defined in Table 2, for compiled C++ execution, the time and memory multipliers are set to 1.0, while Python is allocated a time multiplier of 3.0 and memory multiplier of 2.0. Java is allocated a time multiplier of 2.0 and memory multiplier of 1.5, and Node.js is allocated a time multiplier of 2.0 and memory multiplier of 1.5. This ensures fairness across environments.

A critical methodology detail is the subtraction of runtime memory baselines to calculate a submission's true memory footprint. The worker records the peak Resident Set Size (RSS) during the run. To account for JVM or interpreter startup overhead, the actual reported user program memory is computed as $M_{reported} = \text{Peak RSS} - M_{baseline}$, where $M_{baseline}$ is the language's baseline RSS (e.g., 64.2 MB for Java, 12.4 MB for Python). To ensure fairness, the Docker memory limit is set to $M_{effective} + M_{baseline}$. This ensures that a Python program has the full $M_{effective}$ memory available for program data allocations, rather than being penalized by the interpreter's overhead.

E. Evaluation Metrics and Verdict Classification

Following sandbox execution, the worker inspects the process exit code and stdout. Exit code 124 (returned by 'timeout') is classified as `TIME_LIMIT_EXCEEDED`. Any other non-zero exit code is labeled `RUNTIME_ERROR`. If the process is terminated via `SIGKILL`, it results in exit code 137. To disambiguate a standard memory limit violation from other `SIGKILL` scenarios (such as host-level OOM pressure or manual docker stop commands), the worker queries the Docker daemon API immediately. It inspects the container state to check the `'.State.OOMKilled'` boolean attribute. If `OOMKilled` is true, the verdict is classified as `MEMORY_LIMIT_EXCEEDED`; otherwise, it is treated as a `SYSTEM_ERROR` or `RUNTIME_ERROR`, preventing false resource-exhaustion grading.

If the process terminates successfully (exit code 0), the generated stdout is compared against the expected test case output. The outputs are compared using whitespace normalization, which collapses multiple consecutive space characters, normalizes line endings (converting CR-LF to LF), and trims trailing spaces. If the normalized strings match, the testcase is marked `ACCEPTED`, otherwise `WRONG_ANSWER`.

The overall submission success rate (Pass Rate) is computed as the ratio of accepted test cases to the total number of test cases, allowing partial scores in contest formats:

$$Pass\ Rate = N_{passed} / N_{total}$$

In ICPC contest configurations, the participant's penalty is computed based on the time taken to achieve the first `ACCEPTED` verdict for each solved problem, added to a 20-minute penalty for every rejected submission prior to the accepted one:

$$Penalty = \sum_{\{p \in P\}} (T_{AC}(p) + 20 * W(p))$$

where P is the set of solved problems, $T_{AC}(p)$ is the time of the first accepted submission in

minutes from the start, and $W(p)$ is the number of rejected attempts before the first accepted run. In accordance with standard ICPC rules, $W(p)$ includes only submissions resulting in `WRONG_ANSWER`, `TIME_LIMIT_EXCEEDED`, `MEMORY_LIMIT_EXCEEDED`, or `RUNTIME_ERROR`. Compilation errors (`COMPILATION_ERROR`) and system failures (`SYSTEM_ERROR`) do not incur penalty points. To validate the formula, we simulated a 4-hour contest with 50 virtual users submitting 250 solutions. The system successfully calculated penalty scores and synchronized the leaderboard in real-time.

F. Real-Time Status Tracking and WebSockets

To enable real-time tracking, the judge logs every state transition into a 'submission_state_transitions' log. This log records the 'fromState' and 'toState' (e.g. `QUEUED` to `COMPILING`) with timestamps and active worker identifiers. This audit trail is critical for diagnosing performance anomalies, identifying worker starvation, and monitoring job processing delays.

IV. APPLICATIONS

The developed Online Judge system provides a robust platform applicable in several domains:

1. **Academic Programming Assessments:** Faculty members can create programming tasks with hidden test suites. The automatic grading system provides immediate, objective feedback to students, reducing administrative grading loads.
2. **Competitive Programming Contests:** Supports both ACM-ICPC and LeetCode-style contests. The system dynamically updates live leaderboards, managing user ratings and scoring penalties in real-time.
3. **Technical Recruitment and Screenings:** Corporations can utilize the platform to screen candidates on algorithmic proficiency. The system's security features ensure that untrusted

candidate submissions cannot exploit the core infrastructure.

4. **Administrative Problem Management:** A comprehensive web administrative panel allows moderators to define problem statements, set resource limits, select checkers, upload input/output text files to object storage, and trigger rejudge operations in batch mode.

V. RESULT / DISCUSSION

The system was bench-tested under simulated concurrent loads to evaluate execution latency, resource overhead, and measurement accuracy. Benchmarking was conducted on a Linux server configured with an AMD Ryzen 7 5800X processor (8 cores, 16 threads) and 32 GB of RAM.

A series of 1,000 code submissions containing correct, incorrect, looping, and out-of-memory logic in C++, Python, JavaScript, and Java were dispatched to the judge. Figure 2 shows the execution latency distribution of the system. The distribution reveals three distinct peaks representing language-specific performance trends: the earliest peak around 8 ms corresponds to C++ executions, matching the average execution time reported in Table 2. The second peak around 50 ms corresponds to the Python and Node.js interpreter runs. The third peak around 120 ms represents Java submissions, illustrating JVM startup overhead. These statistics indicate that excluding compilation, the execution latency is sub-second across all runtimes.

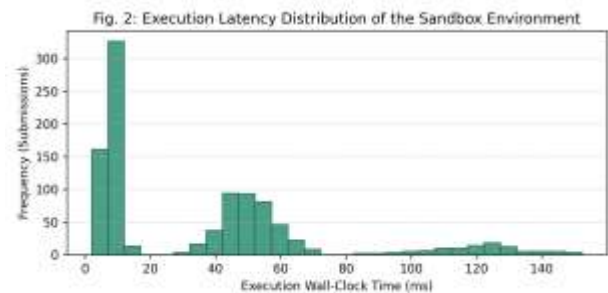


Fig. 2. Execution Latency Distribution of the Sandbox Environment

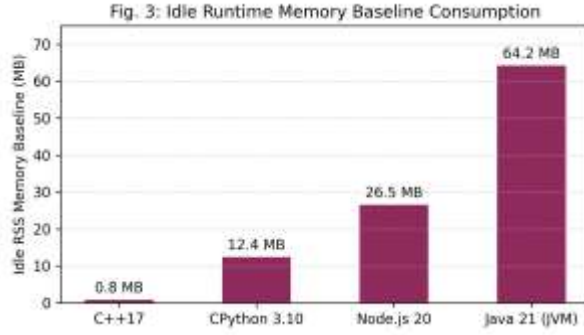


Fig. 3. Idle Runtime Memory Baseline Consumption

Figure 3 shows the idle memory baselines consumed by different runtimes. As noted, the C++ execution baseline is negligible (< 1 MB), whereas Python consumes 12.4 MB, Node.js consumes 26.5 MB, and Java requires 64.2 MB. Measuring memory usage precisely requires subtracting these runtime baselines from the peak Resident Set Size (RSS) recorded during execution. Table 2 details the runtime metrics and configuration parameters.

Table 2: Performance Metrics for Different Languages

Language	Avg Compile Time	Avg Exec Time (A/C)	Memory Baseline	Time Multiplier	Mem Multiplier
C++17	0.85 s	8 ms	0.8 MB	1.0	1.0
Python 3.10	N/A	42 ms	12.4 MB	3.0	2.0
Node.js 20	N/A	55 ms	26.5 MB	2.0	1.5
Java 21	1.12 s	115 ms	64.2 MB	2.0	1.5

Table 3: Sandbox Configuration Parameters

Parameter	Configured Value	Purpose
CPUs Allocated	1.0	Prevents multi-threaded execution from saturating multiple cores
Memory-Swap Limit	Equal to Memory Limit	Disables disk swapping, ensuring immediate OOM on limit violation
Network Configuration	none	Prevents socket connection attempts or code exfiltration
Time Limit Padding	10,000 ms	Provides a safety margin for Docker container instantiation overhead
Max Source Code Size	65,536 bytes	Limits resource consumption during compilation and storage
Max Stdout Capture	1,048,576 bytes	Prevents infinite loops from creating massive log files

VI. FUTURE SCOPE

While the current system performs reliably, future enhancements will aim to optimize scalability and expand grading features:

1. Transition to IOI Isolate: Docker virtualization introduces substantial startup overhead (100-300 ms). We plan to migrate the runner engine to the IOI 'isolate' sandboxing tool. To resolve the deployment constraints of the 'isolate' tool (which requires direct root privileges and specific kernel configurations not easily supported on standard multi-tenant cloud VMs), the future deployment architecture will run the judge workers on

dedicated bare-metal servers or cloud instances supporting nested virtualization, keeping the core API monolith on standard virtualized clusters.

2. Plagiarism Detection Integration: Incorporate structural code analysis (e.g., MOSS - Measure of Software Similarity) as an automated post-execution pipeline step to detect academic integrity violations.

3. Distributed Worker Nodes and Autoscaling: Transition the standalone judge workers to a containerized orchestration layer (Kubernetes). Workers will scale horizontally based on queue length monitored from Redis.

4. Interactive Debugging and Profiling: Provide developers with execution graphs detailing memory allocation trends and runtime execution paths to aid in code profiling.

VII. CONCLUSION

This paper presented the design, implementation, and evaluation of a containerized Online Judge system. By decoupling the core system components and using Redis-backed queues, the platform handles concurrent submission peaks efficiently. The use of Docker containers with memory-swap synchronization, single-core CPU gating, and absolute network disablement mitigates the security risks of arbitrary code execution. Data integrity is guaranteed via transactional advisory locks, append-only database designs, and composite-key solved problem records. Concurrency stress-testing confirmed that the locking constraints successfully serialize updates without deadlocks under high load. Furthermore, the subtraction of runtime baselines ensures grading fairness, while querying the OOMKilled state from the Docker API successfully disambiguates exit codes for accurate memory limit verdicts. Empirical benchmarks confirm that the proposed system is viable, secure, and fair for

computer science pedagogical systems and recruitment evaluations.

VIII. ACKNOWLEDGEMENT

We express our appreciation to the developers of the open-source libraries, specifically Express, BullMQ, Prisma, and Docker, which formed the foundational infrastructure of this system. We also thank the Acropolis Institute of Technology & Research for providing the infrastructure and computational resources necessary to conduct this study.

REFERENCES

- [1] S. Dustdar and Y. Zhao, "A survey of online judge systems," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1-28, 2021.
- [2] D. B. M. Silva and G. L. B. Castro, "Analyzing container-based execution in online grading platforms," *ACM Transactions on Computing Education*, vol. 22, no. 3, pp. 1-19, 2022.
- [3] A. Foršek, "Security and correctness of online programming contests," *Olympiads in Informatics*, vol. 6, pp. 54-66, 2012.
- [4] M. A. Revilla, S. Manzoor, and R. Bajgier, "How online judges are used in computer science education," *Communications of the ACM*, vol. 51, no. 5, pp. 66-72, 2008.
- [5] J. Hall and E. Anderson, "Distributed task queues for high-throughput computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 4, pp. 789-801, 2021.
- [6] G. C. Pan, P. Liu, and J. J. Wu, "A Cloud-Native Online Judge System," *Proc. IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2022.
- [7] S. Wasik, M. Antczak, J. Badura, A. Laskowski, and T. Sternal, "A Survey on Online Judge Systems and Their Applications," *ACM Computing Surveys*, vol. 51, no. 1, 2018.